# User 회원가입 및 로그인 수정본

현재 PartZone 프로젝트의 JWT 기반 로그인 구조를 유지하면서, 수업 프로젝트의 회원가입·입력 검증·중복 처리 흐름을 현재 폴더 구조에 편입한 내용이다.

| 구분 | 정리 |
| --- | --- |
| 적용 범위 | Spring Boot User 도메인, Global 예외, Security, React 회원가입 화면, Oracle USERS ID 시퀀스 |
| 유지한 방식 | JWT + Stateless Security. 수업 프로젝트의 HttpSession 방식으로 변경하지 않음 |
| 핵심 원칙 | 요청은 DTO로 받고, Entity는 DB 모델로 보호하며, 응답은 password 없는 DTO로 보냄 |
| 검증 상태 | 백엔드 compileJava 성공, 프론트 Vite build 성공, 실제 Oracle/API 흐름 검증 완료 |

## 1. 왜 구조를 바꿨는가

• 기존에는 POST /api/users가 User Entity를 그대로 받았다. 요청 JSON이 Entity 내부 필드와 바로 연결되므로 password, createdAt 같은 DB 모델 속성이 API 계약에 섞였다.

• 회원가입 성공 응답도 Entity였기 때문에, 현재는 JsonProperty로 숨겨도 응답 정책을 Entity에 의존하는 구조였다.

• 수정 후에는 SignupRequest가 입력 범위를 제한하고, UserResponse는 외부로 내보낼 필드만 가진다. 따라서 password는 응답 DTO에 아예 존재하지 않는다.

• 로그인은 이미 JWT 기반이므로 세션 로그인으로 복사하지 않았다. LoginResponse의 accessToken 계약을 그대로 보존했다.

## 2. 전체 요청 흐름

```
■■■■
React RegisterPage
→ POST /api/users
→ UserController(@Valid SignupRequest)
→ UserService.signup()
→ BCrypt PasswordEncoder
→ User Entity + UserRepository
→ Oracle USERS
→ UserResponse(■■■■ ■■)
```

```
■■■
React LoginPage
→ POST /api/users/login
→ LoginRequest ■■
→ ■■■ ■■ + BCrypt matches
→ JwtTokenProvider.createToken(userId)
→ LoginResponse(accessToken, userId, email, nickname)
→ sessionStorage ■■
→ ■■■■ ■ GET /api/users/me■ ■■ ■■
```

## 3. DTO: API의 입력과 출력 경계

추가 파일: backend/src/main/java/com/comcheck/comcheck/domain/user/dto/SignupRequest.java

```
public record SignupRequest(
@NotBlank @Email @Size(max = 255) String email,
@NotBlank @Pattern(regexp = "^(?=.*[A-Za-z])(?=.*\\d)\\S{8,20}$") String password,
@NotBlank @Size(max = 50) String nickname
) {}
```

• record는 생성자, getter(accessor), equals/hashCode가 자동 생성되는 불변 DTO다. 요청값을 Service에서 바꾸지 않기 때문에 Setter가 필요 없다.

• @NotBlank는 null·빈 문자열·공백만 있는 값을 막고, @Email은 이메일 모양을, @Size는 DB 컬럼 길이에 맞는 최대 길이를 검사한다.

• 비밀번호 정규식은 영문 1개 이상 + 숫자 1개 이상 + 공백 없음 + 8~20자를 뜻한다.

수정 파일: backend/.../dto/LoginRequest.java

• 로그인 DTO에도 @NotBlank, @Email, @Size를 추가했다. Controller의 @Valid가 Service 실행 전 입력 오류를 400으로 막는다.

추가 파일: backend/.../dto/UserResponse.java

```
public record UserResponse(Long userId, String email,
String nickname, LocalDateTime createdAt) {
public static UserResponse from(User user) { ... }
}
```

• 응답에는 password 필드가 없으므로 실수로 BCrypt 해시가 JSON에 포함되는 일을 구조적으로 차단한다.

## 4. Entity: DB 모델은 Setter로 열어 두지 않는다

수정 파일: backend/src/main/java/com/comcheck/comcheck/domain/user/entity/User.java

• JPA용 protected 기본 생성자를 둔다. JPA는 DB 행을 User 객체로 복원할 때 이 생성자를 사용한다.

• 회원 생성은 public User(email, encodedPassword, nickname)으로만 한다. 평문 password를 직접 넣는 생성 경로를 만들지 않는다.

• 비밀번호 변경은 changePassword(encodedPassword) 메서드로만 처리한다. setter를 없애면 어떤 값이 들어가는지 Service에서 추적하기 쉽다.

• @PrePersist는 INSERT 직전에 createdAt이 비어 있으면 현재 시각을 설정한다.

실무 포인트: Entity는 Controller 요청 본문이나 응답 본문으로 직접 사용하지 않는 것이 안전하다. 이를 API Entity Exposure 또는 Mass Assignment 방지라고 부른다.

## 5. Service: 실제 비즈니스 규칙이 있는 곳

수정 파일: backend/src/main/java/com/comcheck/comcheck/domain/user/service/UserService.java

```
String encodedPassword = passwordEncoder.encode(request.password());
User savedUser = userRepository.save(
new User(email, encodedPassword, nickname)
);
return UserResponse.from(savedUser);
```

• PasswordEncoder는 SecurityConfig의 Bean으로 등록한 BCryptPasswordEncoder를 주입받는다. Service가 new로 직접 만들지 않아 교체·테스트가 쉽다.

• existsByEmail / existsByNickname으로 먼저 확인해 사용자에게 409 Conflict와 이해 가능한 문구를 준다.

• 로그인은 passwordEncoder.matches(평문, 저장된 해시)로 비교한다. DB의 해시 문자열을 평문과 == 또는 equals로 비교하면 안 된다.

• findUserById는 없을 때 ResponseStatusException(404)을 던진다. Controller가 null을 받고 매번 if문을 작성하지 않아도 된다.

• @Transactional(readOnly = true)를 클래스 기본값으로 두고 signup, changePassword, deleteUser에만 쓰기 Transaction을 붙였다.

## 6. Controller와 Global 오류 응답

수정 파일: backend/.../controller/UserController.java

• POST /api/users는 @Valid @RequestBody SignupRequest를 받으며 성공 시 201 Created + UserResponse를 반환한다.

• POST /api/users/login도 @Valid LoginRequest를 사용하며 accessToken을 포함한 LoginResponse를 유지한다.

• GET /api/users/me와 DELETE /api/users/{userId}는 JWT principal의 userId로 본인 소유 여부를 검사한다.

수정 파일: backend/.../global/exception/GlobalExceptionHandler.java

• MethodArgumentNotValidException을 잡아 validation 오류를 ApiErrorResponse로 통일했다. errors에는 email, password, nickname처럼 필드별 문구가 들어간다.

추가 파일: backend/.../global/exception/SecurityErrorResponseWriter.java

• Security Filter 단계의 401/403은 ControllerAdvice까지 도달하지 않는다. 이 Writer가 토큰 없음·권한 없음·변조 토큰에도 같은 ApiErrorResponse 필드를 쓰게 한다.

## 7. Security와 프론트 연결

수정 파일: backend/.../config/SecurityConfig.java

• PasswordEncoder Bean을 등록했다.

• POST /api/users, POST /api/users/login은 로그인 전에도 접근 가능하다. 나머지 보호 API는 JWT를 요구한다.

• 토큰 없음은 401 UNAUTHORIZED, 다른 사용자 계정 접근은 403 FORBIDDEN의 공통 JSON으로 반환한다.

수정 파일: backend/.../domain/user/security/JwtAuthenticationFilter.java

• 잘못된 Bearer token은 SecurityContext를 비우고 SecurityErrorResponseWriter로 401을 응답한다.

수정 파일: frontend/src/api/client.js

• 서버 오류 JSON의 message와 errors를 ApiError에 보관한다. 204 No Content도 JSON 파싱 오류 없이 null로 처리한다.

수정 파일: frontend/src/api/users.js, frontend/src/pages/RegisterPage.jsx

• 회원가입 버튼은 이제 실제 POST 요청을 보낸다. 요청 중에는 버튼을 비활성화하고, 400/409은 서버가 준 필드 오류 또는 메시지로 표시한다.

• 프론트 정규식은 UX용 빠른 검사다. 서버 DTO 검증이 최종 방어선이므로 Postman·curl 같은 우회 요청도 막는다.

## 8. Oracle 변경

확인한 USERS 스키마: user_id NUMBER, email VARCHAR2(255), password VARCHAR2(255), nickname VARCHAR2(50), phone_number VARCHAR2(20), created_at DATE

• User Entity는 SEQ_USERS를 사용하도록 설정돼 있었지만 실제 DB에 해당 시퀀스가 없어 회원가입 시 ORA-02289가 발생했다.

```
CREATE SEQUENCE SEQ_USERS
START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
```

• USERS가 0건인 상태를 확인한 뒤 SEQ_USERS를 1부터 생성했다. 현재 @SequenceGenerator와 이름이 일치한다.

• email/nickname에는 UNIQUE 제약이 아직 없다. 현재 Service 중복 검사는 동시 요청 경쟁 상태(Race Condition)를 완전히 막지 못한다.

• 운영 전 권장 순서: 기존 중복 데이터 확인 → 중복 제거 → UNIQUE 제약 또는 UNIQUE INDEX 추가 → DataIntegrityViolationException을 Global Handler에서 409로 처리.

## 9. API 사용 예시

회원가입 요청

```
POST /api/users
Content-Type: application/json

{
"email": "gyul01@example.com",
"password": "gyul1234",
"nickname": "gyul01"
}
```

회원가입 성공 응답 (201)

```
{
"userId": 1,
"email": "gyul01@example.com",
"nickname": "gyul01",
"createdAt": "2026-10-14T20:00:00"
}
```

검증 오류 응답 (400)

```
{
"status": 400,
"code": "BAD_REQUEST",
"message": "[redacted].",
"timestamp": "...",
"errors": {
"email": "[redacted]."
}
}
```

로그인 요청과 인증 헤더

```
POST /api/users/login
{ "email": "gyul01@example.com", "password": "gyul1234" }

GET /api/users/me
Authorization: Bearer {accessToken}
```

## 10. 실제 검증 결과와 다음 점검

| 검증 항목 | 결과 |
| --- | --- |
| backend ./gradlew compileJava | 성공 |
| frontend npm run build | 성공 |
| POST /api/users | 201 회원가입 성공 |
| 같은 회원가입 재요청 | 409 CONFLICT 공통 오류 JSON 확인 |
| 잘못된 DTO 요청 | 400 BAD_REQUEST + errors 필드 확인 |
| 로그인 → Bearer /me | 200 및 password 미노출 확인 |
| 토큰 없는 /me | 401 UNAUTHORIZED 공통 오류 JSON 확인 |

| 검증 항목 | 결과 |
| --- | --- |
| 검증용 계정 | 각 테스트 후 Oracle에서 삭제 완료 |

• 다음에 볼 것: DB UNIQUE 제약과 애플리케이션 중복 검사 역할 차이, 그리고 Race Condition.

• 10배 규모에서는 중복 검사만 믿지 말고 DB UNIQUE를 최종 보루로 두고, 로그인 시도 제한·Refresh Token·비밀번호 변경 DTO 검증을 추가해야 한다.